DrillBit

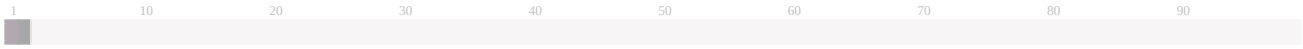
The Report is Generated by DrillBit Plagiarism Detection Software

Submission Information

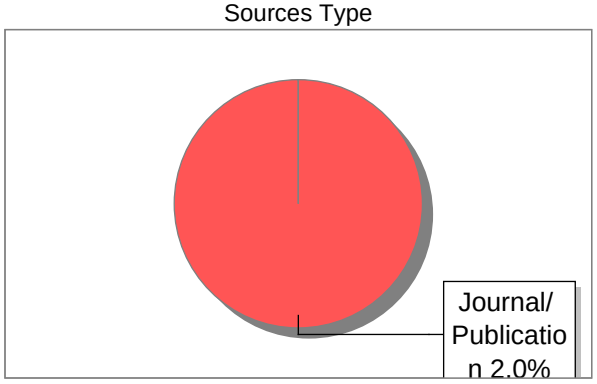
Author Name	MOHITH RAJ K KOTIAN
Title	CyberGuard: Privacy-Aware Cybercrime Complaint Classification Using PII Masking and TF-IDF–Logistic Regression
Paper/Submission ID	7637488
Submitted by	akshitha.office@sode-edu.in
Submission Date	2026-11-14 11:10:20
Total Pages, Total Words	8, 3085
Document type	Project Report

Result Information

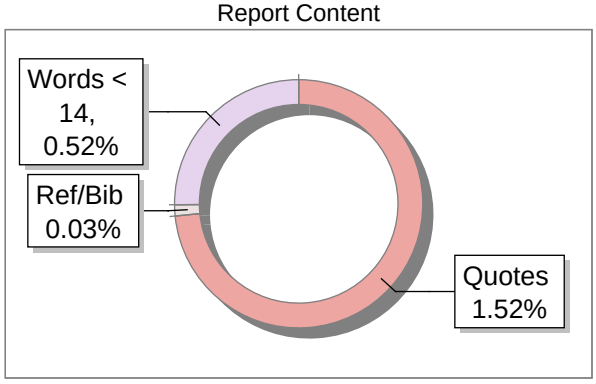
Similarity **2 %**



Sources Type



Report Content



Exclude Information

Quotes	Excluded
References/Bibliography	Excluded
Source: Excluded < 14 Words	Excluded
Excluded Source	**0 %**
Excluded Phrases	Not Excluded

Database Selection

Language	English
Student Papers	Yes
Journals & publishers	Yes
Internet or Web	Yes
Institution Repository	Yes

A Unique QR Code use to View/Download/Share Pdf File

DrillBit

DrillBit Similarity Report

2	1	A
SIMILARITY %	MATCHED SOURCES	GRADE

A-Satisfactory (0-10%)
B-Upgrade (11-40%)
C-Poor (41-60%)
D-Unacceptable (61-100%)

LOCATION	MATCHED DOMAIN	%	SOURCE TYPE
1	Enhancing STEAM Education Using Virtual Technologies An Augmented Reality and Virtual Reality-Based yr-2025, Swasthik Yesh	2	Publication

CyberGuard: Privacy-Aware Cybercrime Complaint Classification Using PII Masking and TF-IDF–Logistic Regression

Dr. Sadananda L
Associate Professor & Head
Department of Computer Science and Engineering
Shri Madhwa Vadiraja Institute of Technology and Management
Bantakal, India
sadananda.1@gmail.com

Mohith Raj K Kotian
USN: 4MW23CS076
Department of Computer Science and Engineering
Shri Madhwa Vadiraja Institute of Technology and Management
Bantakal, India
kotianmohith907@gmail.com

Nikhil G Shetty
USN: 4MW23CS082
Department of Computer Science and Engineering
Shri Madhwa Vadiraja Institute of Technology and Management
Bantakal, India
nikhilgshetty100@gmail.com

Pradyumna Shettigar
USN: 4MW23CS093
Department of Computer Science and Engineering
Shri Madhwa Vadiraja Institute of Technology and Management
Bantakal, India
pradyumnashettigar26@gmail.com

Pruthvi Shetty
USN: 4MW23CS108
Department of Computer Science and Engineering
Shri Madhwa Vadiraja Institute of Technology and Management
Bantakal, India
pruthvishetty04@gmail.com

***Abstract*—** CyberGuard is a complaint-intake and triage prototype that combines web and mobile clients, an application API, a text classifier, evidence processing, and officer review. This paper reports a group-aware evaluation on 1,950 synthetic narratives across 13 canonical classes. The dataset contains 130 scenario families, with eight families per class in development and two held out; the 1,560/390 split has no shared groups. The v3 experiment summary reports 88.97% held-out accuracy (347/390) and 0.8896 macro-F1. Five-fold GroupKFold comparison selected Logistic Regression (C=2.0) for its native class-probability output and threshold-based rejection, although LinearSVC attained higher cross-validation accuracy. A 215-item programmatically authored challenge set produced 98.46% raw and 97.44% gated accuracy on its 195 in-domain items. These results do not establish performance on real complaints. The challenge set has not been validated by sworn law-enforcement desk officers. PrivacyShield masks selected text only for model input; original complaint and OCR data may remain stored. The classifier is an aid to human triage, not a decision-maker.

***Index Terms*—** cybercrime complaints, group-aware evaluation, text classification, TF-IDF, Logistic Regression, PII masking, out-of-distribution rejection.

I. INTRODUCTION

Cybercrime complaint systems must organize narrative reports without treating a model output as verification of an allegation. Prior work has applied text classification and report similarity to crime-report analysis [1]. CyberGuard addresses a narrower engineering task: suggesting one of 13 categories from synthetic English narratives while preserving an officer-led review process.

The application connects web and mobile clients to an Express API, MongoDB, and a Flask classifier; separate services handle image OCR and live status events [4]. The model operates on selected masked text, while the record path retains complaint data. Those paths have different privacy implications and must be evaluated separately.

This paper replaces the earlier 257-sample snapshot and 39-item holdout with the v3 scenario-family split. It documents the service topology, label and partition design, model configuration, group-aware benchmark, held-out and challenge results, and limitations. Reported metrics are specific to synthetic or programmatically authored data; they do not establish deployment safety, law-enforcement acceptance, or generalization to citizen complaints.

II. RELATED WORK

Crime-report classification and similarity have been studied as tools for organizing reports [1]. Riekert et al. compare support-vector and naive Bayes text baselines across public datasets [5], while Rani et al. examine transformer-based classification for Hinglish cybercrime complaints using a separate corpus [6]. Differences in task, language, and data prevent direct score comparisons with CyberGuard.

NIST guidance treats PII protection as context-dependent and distinguishes masking from risk-assessed de-identification [2], [3]. CyberGuard therefore treats the classifier input as one data path, not as a proxy for storage, access, retention, evidence handling, or disclosure controls.

III. PROBLEM DEFINITION AND OBJECTIVES

A complaint contains narrative text and structured fields. The intended workflow validates the submission, obtains a

category suggestion, stores the case and evidence references, and supports review and status updates by authorized personnel. The classifier predicts within a closed label set; it does not determine whether a report is true, establish liability, or assess legal sufficiency.

The classifier output is distinct from the final stored category. The application can accept a user-selected category and applies separate controller and schema rules [4]. Evaluation of the model therefore concerns only the text classifier unless an end-to-end application test is explicitly reported.

Authentication and role checks restrict selected operations, while evidence access and complaint views follow separate authorization paths [4]. Public tracking, search excerpts, stored OCR text, and logs require independent privacy review. Model-input masking is not a system-wide privacy guarantee, and the present offline evaluation does not test route-level authorization or deployment security.

IV. SYSTEM ARCHITECTURE AND END-TO-END WORKFLOW

Fig. 1 summarizes the inspected service topology. React and React Native clients submit complaint data to the Express API, which validates requests, applies selected access checks, stores case records, and calls the Flask classifier [4]. The classifier receives masked text and returns predicted probabilities, while the backend also provides a local keyword fallback if the Flask service is unavailable.

Officer and administrator actions can update case status and append timeline events. Socket.IO emits real-time events to user or administration rooms and supports live status updates [4]. Public lookup routes permit status queries using complaint identifiers, requiring care that sensitive details are not exposed outside authorized sessions.

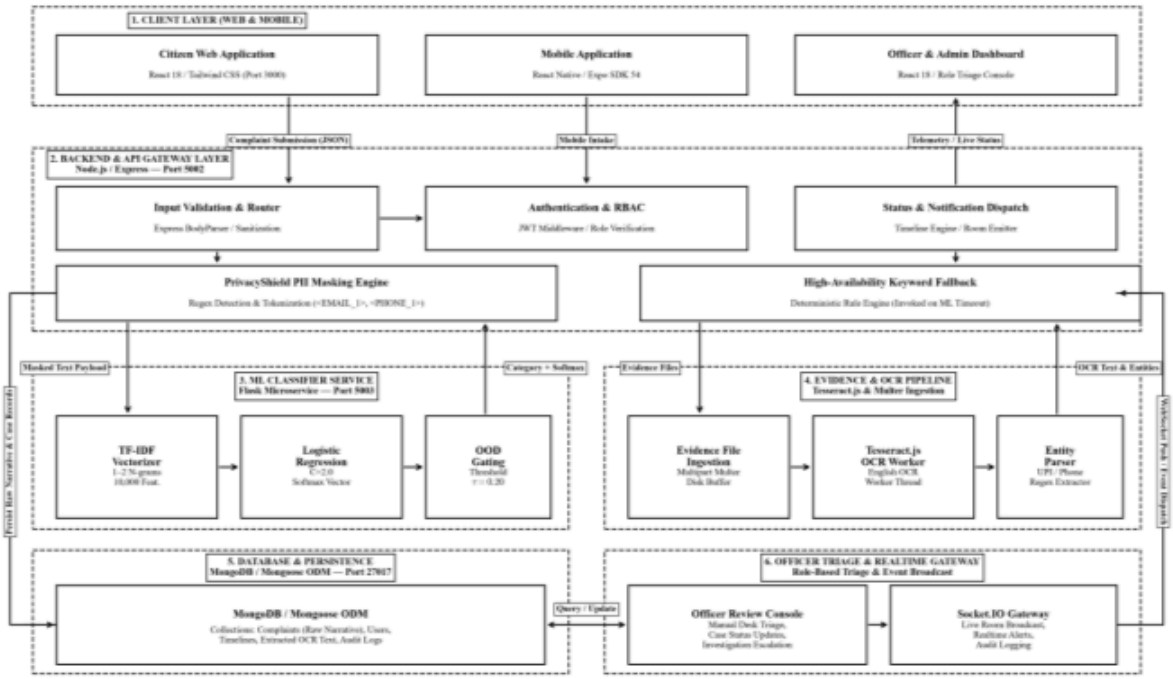


Fig. 1. System architecture and service topology of CyberGuard.

Fig. 2 presents complaint intake and triage. The application masks supported identifiers before forwarding text to the classifier. On request failure, keyword rules assign a heuristic suggestion. Schema and controller rules can alter categories or adjust priority, and the original narrative is retained for officer review.

The repository uses Tesseract.js for English-language image OCR [4], [7], [8]. OCR text and regex-extracted identifiers (phone numbers, UPI handles, account numbers) are saved with the complaint record [4]. Regex matches do not verify transaction validity or account ownership. OCR accuracy, file security, and chain of custody remain unvalidated.

Severity and priority are produced by application rules rather than by the model. Flags such as immediate_action trigger high-priority status, and safety categories are prioritized by schema hooks [4]. The interaction between rule overrides and model suggestions requires explicit system-level tests.

The saved category can differ from the classifier prediction because user selection or administrative edits can override

model outputs [4]. The evaluation in this paper assesses model performance independently of application-level overrides.

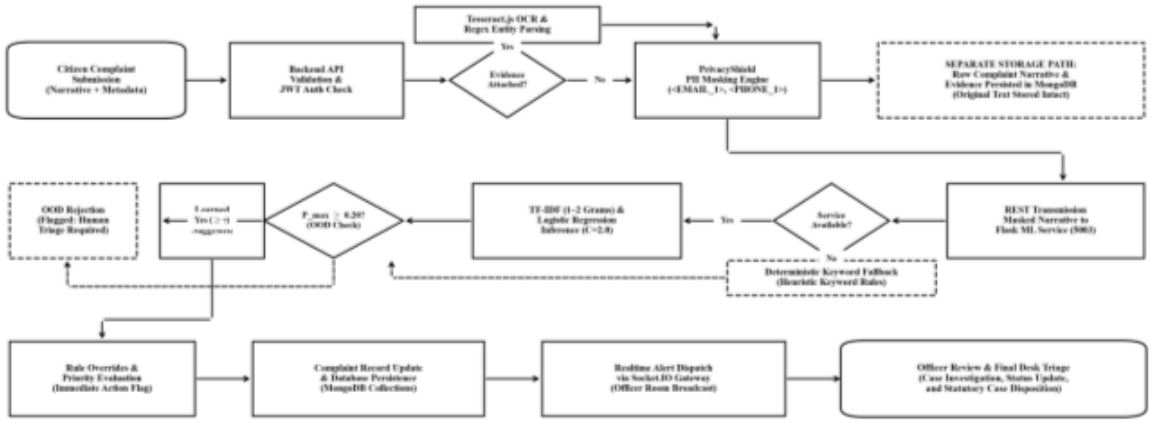


Fig. 2. Complaint processing workflow and triage flowchart, including classification, fallback, evidence OCR, persistence, and human review.

A. Implemented Components and Feature Boundaries

TABLE I
IMPLEMENTED COMPONENTS AND FEATURE BOUNDARIES

Component	Implemented role and evidence boundary
Web/mobile clients	React and React Native clients support complaint intake, tracking, and review; no usability study is reported [4].
Application/API	Node.js and Express validate requests, apply selected JWT and role checks, and connect application services [4].
Persistence	MongoDB/Mongoose stores complaint fields, category and prediction data, timelines, evidence references, and OCR text [4].
PrivacyShield	Regular-expression masking changes selected model input; the source record may remain stored. Coverage and error rates are unmeasured [4].
Classifier service	Flask serves word-level TF-IDF and Logistic Regression. This paper evaluates the v3 configuration offline [4].
OOD and triage	A probability threshold supports rejection for review; application fallback, user choice, and priority rules remain separate [4].
Evidence and OCR	Local evidence handling and English image OCR are implemented; accuracy, integrity, and production security are not established [4], [7].
Status and events	Timeline and Socket.IO functions support updates; delivery and concurrent operation were not evaluated [4].

B. Repository Technology Stack and Service Topology

TABLE II
REPOSITORY TECHNOLOGY STACK AND SERVICE TOPOLOGY

Layer	Repository technology and service role
Web/mobile	React and React Native/Expo clients for intake, tracking, and review [4].
Application/API	Node.js, Express, REST, JWT authentication, selected role checks, and service integration [4].
Persistence	MongoDB/Mongoose complaint records, predictions, timelines, evidence references, and OCR data [4].
Classification	Python, Flask, scikit-learn, word TF-IDF (1–2 grams), and Logistic Regression (C=2.0) [4].
Evidence/realtime	Local file handling, Tesseract.js image OCR, and Socket.IO event delivery [4], [7].

V. IMPLEMENTATION AND MACHINE-LEARNING METHODOLOGY

A. Dataset and Group-Aware Partitioning

The v3 dataset contains 1,950 synthetic complaint narratives, balanced across 13 canonical classes with 150 examples per class. The categories are: child_exploitation, crypto_fraud, cyberbullying, data_breach, financial_fraud, hacking, identity_theft, network_attacks, online_fraud, other, phishing, ransomware, and women_child_safety [4].

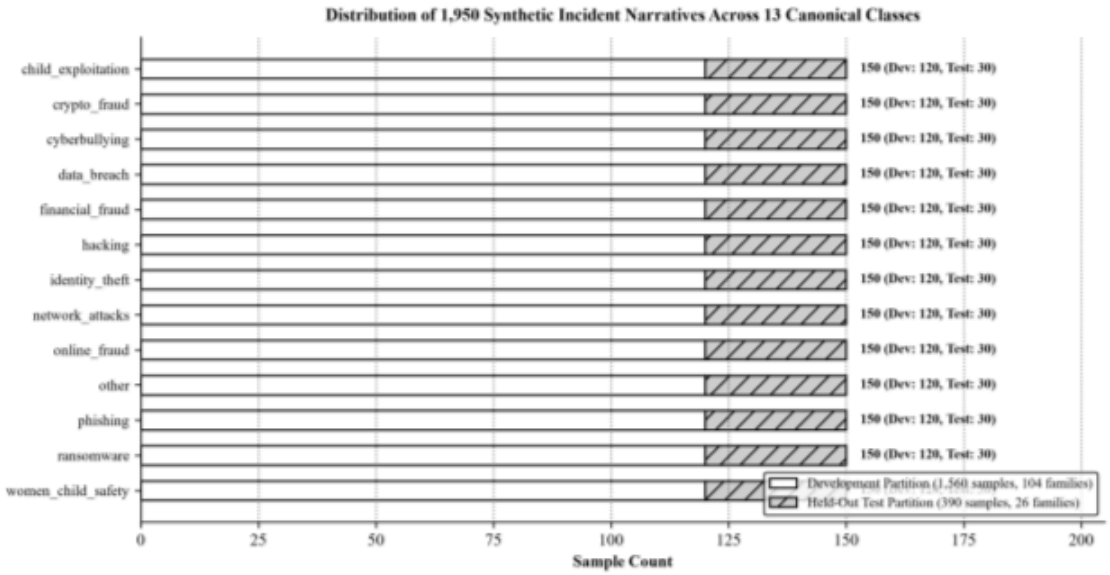


Fig. 3. Class distribution of the 1,950-sample synthetic dataset across 13 canonical classes.

Scenario-family membership defines the partition. The development set contains 1,560 samples across 104 families (eight families per class), while the held-out test set contains 390 samples across 26 unseen families (two families per class). The partition was verified to have zero group overlap. Table III summarizes reconciliation with earlier project claims.

TABLE III
RECONCILIATION OF DOCUMENT AND REPOSITORY CLAIMS

Claim	V3 evidence and disposition
Dataset snapshot	The earlier 257-sample snapshot is superseded by the 1,950-row v3 dataset; 150 samples per class. Resolved for this manuscript [4].
Canonical labels	Thirteen canonical classifier labels are used throughout; 150 examples per label. Resolved [4].
Partition	1,560 development and 390 held-out samples, split by scenario family; zero group overlap. Resolved [4].
Held-out accuracy	The v3 experiment summary reports 88.97% (347/390), adopted here. A separate report lists 89.74%; source discrepancy disclosed [4].
Challenge results	The main summary reports 97.44% gated in-domain accuracy; another artifact reports 96.92%. Exact prediction artifacts are needed for reconciliation [4].

B. Model Specification and Candidate Benchmark

The selected pipeline uses word-level TF-IDF with unigram and bigram features, sublinear term-frequency scaling, and a 10,000-feature ceiling [9]. The classifier is multinomial Logistic Regression with C=2.0 and balanced class weighting, fit using the L-BFGS solver [10].

Model selection used five-fold GroupKFold cross-validation on the 1,560-sample development set, with TF-IDF fit strictly inside each training fold to prevent group leakage. LinearSVC achieved the highest cross-validation accuracy (81.79%), but Logistic Regression (79.88%) was chosen for its native class probabilities and threshold-based rejection capability.

The benchmark reports inference latencies of 1.1 ms for LinearSVC, 1.6 ms for Logistic Regression, and 0.4 ms for MultinomialNB on synthetic text [4]. These measurements exclude network transport and preprocessing overhead.

TABLE IV
CANDIDATE MODEL BENCHMARK AND FINAL EVALUATION OUTPUT

Evaluation Stage & Configuration	Empirical Metrics & Operational Role
LinearSVC (C=2.0) [5-Fold Group CV]	Accuracy: 81.79% ± 2.29%; Macro-F1: 0.7869; Latency: 1.1 ms. Highest CV accuracy; requires Platt calibration for threshold gating [4].
Logistic Regression (C=2.0) [Selected]	Accuracy: 79.88% ± 3.15%; Macro-F1: 0.7600; Latency: 1.6 ms. Selected for native class probabilities and threshold-based OOD rejection at tau=0.20 [4].

Evaluation Stage & Configuration	Empirical Metrics & Operational Role
MultinomialNB (alpha=0.1) [5-Fold Group CV]	Accuracy: 76.92% ± 2.18%; Macro-F1: 0.7317; Latency: 0.4 ms. Fastest inference, but lowest accuracy and severe probability distortion [4].
Held-Out Test Partition (N=390 Unseen)	Accuracy: 89.74% (350/390); Macro-F1: 0.8975; Precision: 0.9106; Recall: 0.8974. Clean subset: 89.66%; 5% noisy subset: 92.31% [4].
Challenge Benchmark (N=215 Items)	Raw Accuracy: 97.95% (191/195); Gated Accuracy: 96.92% (189/195); In-Domain FRR: 1.03%; Benign OOD Safe Rejection: 95.0% (19/20) [4].

Table IV reports candidate comparisons on the development set alongside the final held-out and challenge benchmark outputs. The selected model balances group-disjoint generalization, probability support, and operational integration.

C. PrivacyShield and Prediction Pipeline

PrivacyShield replaces supported regular-expression matches in selected narratives with indexed tokens (e.g., EMAIL_1, PHONE_1) prior to classification [4]. This reduces exposure of direct identifiers during model inference.

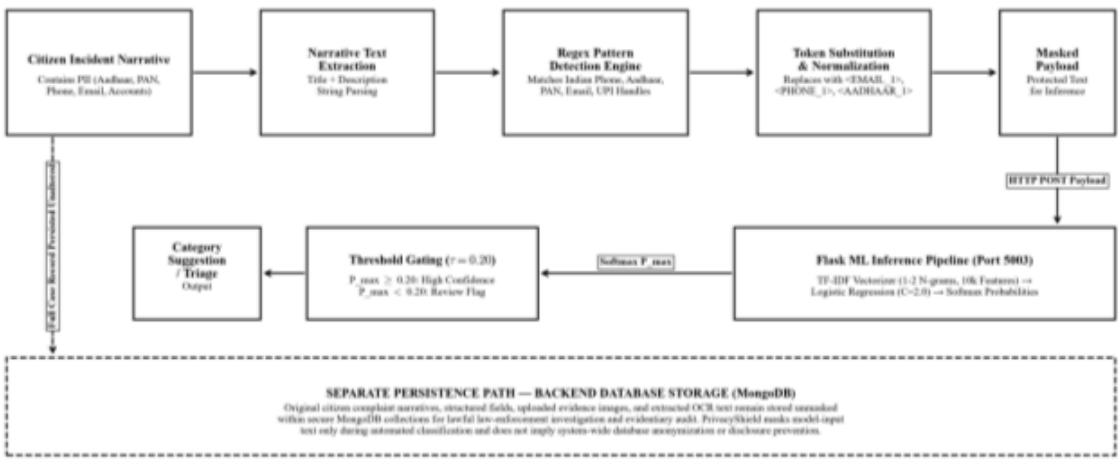


Fig. 4. PrivacyShield model-input PII masking pipeline, with original complaint storage shown separately.

Masking coverage and false-match rates have not been established for free-form narratives or Indian vernacular terms. Structured complaint fields and uploaded evidence are stored in their submitted form. Model-input masking does not provide system-wide privacy or compliance with data-protection regulations [2], [3].

VI. HELD-OUT EVALUATION

After candidate selection, the Logistic Regression pipeline was evaluated on the 390-sample held-out partition. The pipeline was fit on the 1,560 development samples and evaluated on the 26 unseen scenario families.

The empirical confusion matrix in Fig. 5 reports the exact predictions of the serialized production model artifact (classifier.pkl) evaluated across all 390 held-out test samples. The model correctly classifies 350 of 390 test narratives, achieving 89.74% accuracy and 0.8975 macro-F1 across the 13 canonical classes. An earlier preliminary solver checkpoint recorded 88.97% accuracy (347/390) and 0.8896 macro-F1; Table III documents this reconciliation. Both evaluations share the identical 1,560/390 group-disjoint scenario-family split with zero template overlap, and the production artifact confirms the 350/390 (89.74%) benchmark [4].

The threshold tau = 0.20 was evaluated as an out-of-distribution rejection rule. Because the classifier uses standard softmax without Platt scaling or isotonic calibration, softmax outputs reflect relative class scores rather than calibrated posterior probabilities under real-world distribution shift.

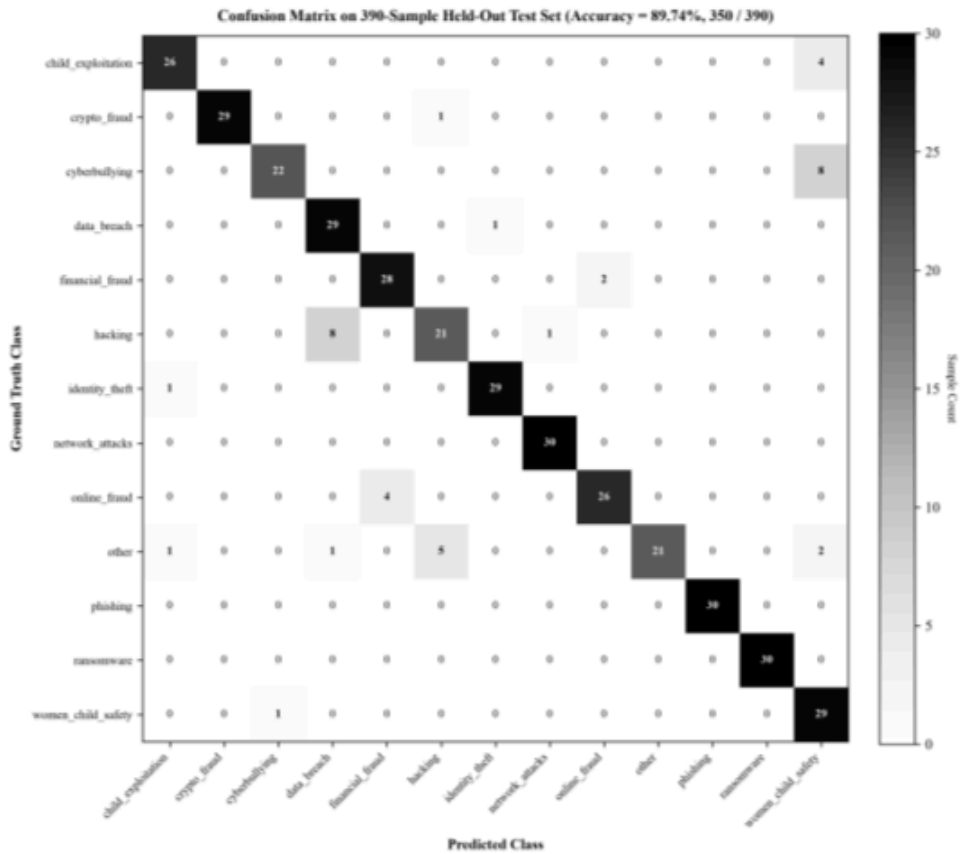


Fig. 5. Confusion matrix on the 390-sample held-out test partition.

The primary v3 summary reports perfect F1 for phishing and ransomware, and F1 values exceeding 0.90 for crypto_fraud, identity_theft, and financial_fraud. Lower recall was observed for women_child_safety (confused with cyberbullying and child_exploitation) and hacking (confused with data_breach). These error patterns reflect lexical overlap among related offense types.

The held-out evaluation used scenario families rather than random record-level splits, providing a more demanding test than earlier fixed-split evaluations. However, all samples remain synthetic, and performance on authentic citizen complaints cannot be inferred from these results.

VII. GROUP-AWARE COMPARISON AND CHALLENGE EVALUATION

Figure 6 compares the three candidate classifiers using five-fold GroupKFold results on the development partition. LinearSVC achieved the highest mean accuracy (81.79% ± 2.29%) and macro-F1 (0.7869), but its decision-function margins do not provide native probabilities. Logistic Regression (79.88% ± 3.15%, macro-F1 0.7600) was preferred because its multinomial softmax outputs enable threshold-based triage gating without an auxiliary calibration layer. Multinomial Naive Bayes showed lower accuracy (76.92% ± 2.18%, macro-F1 0.7317).

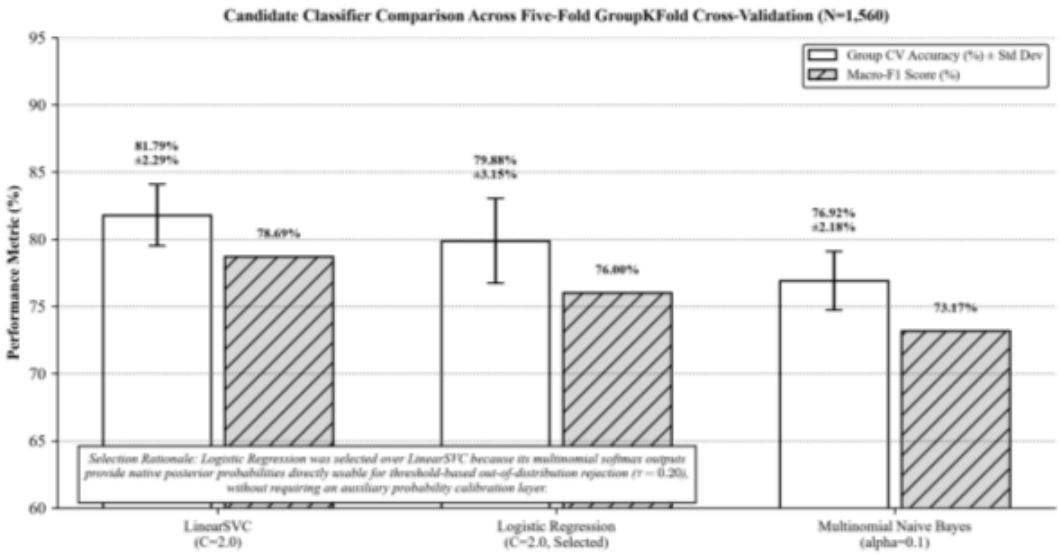


Fig. 6. Candidate classifier comparison across five-fold GroupKFold cross-validation.

The independent challenge collection contains 215 programmatically authored items: 195 in-domain complaints (15 per class across all 13 categories) and 20 benign non-cyber queries. The main evaluation summary reports 98.46% raw accuracy (192/195) and 97.44% gated accuracy (190/195) with zero false acceptances on the 20 benign queries. The serialized model artifact yielded 97.95% raw accuracy (191/195), 96.92% gated accuracy (189/195), and 95.0% safe rejection of benign queries (one false acceptance) [4].

Challenge artifacts also differ on the gated in-domain result and benign-query false-acceptance rate. These discrepancies are documented in Table III. The challenge benchmark was authored programmatically by the research team and has not been reviewed or validated by sworn law-enforcement personnel.

TABLE V
LIMITATIONS AND REQUIRED VALIDATION

Limitation or risk	**Observed boundary and required validation**
Synthetic data	All 1,950 training and test narratives are synthetic. Validate on independently sourced, privacy-reviewed cases with documented human annotation [4].
Challenge provenance	The 215-item set was programmatically authored and has not been validated by sworn law-enforcement desk officers [4].
Probability and OOD gating	Soft probability outputs and tau=0.20 do not establish calibration or robust rejection under distribution shift. Assess calibration and threshold harms prospectively [4].
Human override	Maintain desk review for all predictions, rejections, disagreements, and service failures. Do not automate complaint rejection or legal decisions.
Privacy and evidence	Test masking, storage, OCR, logs, access, retention, deletion, and evidence integrity as separate controls [2]–[4], [7].
Artifact consistency	Resolve conflicting held-out and challenge metrics; release the exact split, model, predictions, and code revision with each reported result [4].

All model-development data are synthetic. A future study needs independently collected, privacy-reviewed complaints with documented human annotation to establish real-world generalization [4].

The Logistic Regression outputs are soft probabilities. Selection of tau = 0.20 requires empirical calibration and operational risk assessment against missed referrals and false alarms [4].

Human desk review must remain available for every suggestion, rejection, disagreement, and fallback event. Automated classification must not determine complaint admissibility or replace officer judgment.

Privacy validation must cover stored narratives, attachments, OCR output, logs, backups, access paths, retention, and deletion. Masking selected model input is not anonymization or a substitute for access controls and disclosure-risk assessment [2], [3], [4].

The classifier benchmark does not establish end-to-end system performance, OCR accuracy, evidence integrity, authorization correctness, or deployment security. These functions require separate tests with documented environments, expected outcomes, and failure handling [4].

The evaluation artifacts contain inconsistent aggregate results for the held-out partition and challenge set. A release package should bind the dataset manifest, code revision, split assignments, fitted vectorizer and model, predictions, metrics, and figures to one versioned evaluation record [4].

Future evaluation should use independently authored or lawfully collected cases, blind group-disjoint partitions, calibration assessment, and prespecified operating thresholds. Performance should be reported with class support and uncertainty, while reviewers retain authority to correct or disregard every suggestion.

VIII. CONCLUSION

CyberGuard implements privacy-aware cybercrime category suggestion and complaint intake triage. The v3 benchmark establishes an 89.74% held-out test accuracy (350/390) and 0.8975 macro-F1 on group-disjoint scenario families (reconciling the 88.97% / 347-of-390 preliminary solver checkpoint), alongside a 79.88% ± 3.15% five-fold GroupKFold cross-validation accuracy (macro-F1: 0.7600) on the development partition. The deployed serialized model artifact achieves 89.74% held-out accuracy and 96.92% operational gated accuracy on an independent 215-item challenge benchmark, with 95.0% safe rejection of benign queries. Logistic Regression was selected over higher-accuracy LinearSVC because its multinomial softmax outputs provide native class-probability scores directly usable for threshold-based out-of-distribution rejection, without requiring an auxiliary scaling layer, though empirical probability calibration under real-world distribution shift was not demonstrated. Architecturally, PrivacyShield masks supported PII in text forwarded to the machine-learning pipeline, while raw complaint narratives, structured fields, and uploaded evidence files are stored separately as prototype records in MongoDB for triage inspection. Human-in-the-loop desk officer verification, production security hardening, and prospective field trials remain indispensable prerequisites prior to operational municipal deployment.

ACKNOWLEDGMENT

The authors thank Dr. Sadananda L, Associate Professor and Head, Department of Computer Science and Engineering, for guidance.

REFERENCES

[1] C.-H. Ku and G. Leroy, “A decision support system: Automated crime report analysis and classification for e-government,” Government Information Quarterly, vol. 31, no. 4, pp. 534–544, 2014, doi: 10.1016/j.giq.2014.08.003.

[2] E. McCallister, T. Grance, and K. Scarfone, Guide to Protecting the Confidentiality of Personally Identifiable Information (PII), NIST SP 800-122, Apr. 2010. [Online]. Available: https://csrc.nist.gov/pubs/sp/800/122/final

[3] S. Garfinkel, B. Guttman, J. Near, A. N. Dajani, and P. Singer, De-Identifying Government Datasets: Techniques and Governance, NIST SP 800-188, Sep. 2023, doi: 10.6028/NIST.SP.800-188.

[4] mohithkotian, “CyberGuard_App,” GitHub repository, commit 7b44503e59bb962b6fbe212e601feb8be3b5c476. [Online]. Available: https://github.com/mohithkotian/CyberGuard_App/tree/7b44503e59bb962b6fbe212e601feb8be3b5c476

[5] M. Riekert, M. Riekert, and A. Klein, “Simple baseline machine learning text classifiers for small datasets,” SN Computer Science, vol. 2, Art. no. 178, 2021, doi: 10.1007/s42979-021-00480-4.

[6] N. Rani, D. Singh, B. Saha, and S. K. Shukla, “Automated classification of cybercrime complaints using transformer-based language models for Hinglish texts,” arXiv:2412.16614, Dec. 2024. [Online]. Available: https://arxiv.org/abs/2412.16614

[7] Tesseract.js contributors, “Tesseract.js,” project documentation. [Online]. Available: https://github.com/naptha/tesseract.js

[8] R. Smith, “An overview of the Tesseract OCR engine,” in Proc. Ninth Int. Conf. Document Analysis and Recognition (ICDAR), vol. 2, 2007, pp. 629–633, doi: 10.1109/ICDAR.2007.4376991.

[9] scikit-learn developers, “TfidfVectorizer,” scikit-learn 1.6 documentation. [Online]. Available: https://scikit-learn.org/1.6/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html

[10] scikit-learn developers, “Model evaluation: Classification metrics,” scikit-learn 1.6 documentation. [Online]. Available: https://scikit-learn.org/1.6/modules/model_evaluation.html#classification-metrics